

Capability Maturity Model (CMM)

The **Capability Maturity Model (CMM)** was developed by **SEI (Software Engineering Institute, Carnegie Mellon University)** to assess and improve the **software process maturity** of organizations.

5 Levels of CMM Maturity

1. Level 1 – Initial (Chaotic)

- Process is ad-hoc, unpredictable.
- Success depends on individual effort, not organization.
- No defined standards.

2. Level 2 – Repeatable

- Basic project management practices are established.
- Past successes can be repeated.
- Example: Cost estimation, schedule tracking.

3. Level 3 – Defined

- Processes are documented and standardized across the organization.
- Training programs exist.
- Engineering and management activities are integrated.

4. Level 4 – Managed

- Processes are measured and controlled quantitatively.
- Statistical techniques are applied.
- Focus on quality and productivity measurement.

5. Level 5 – Optimizing

- Continuous process improvement is the focus.
- Defect prevention, process optimization, and innovation are key.

Software Quality

Software Quality = The degree to which software meets customer requirements, expectations, and standards.

Key Dimensions of Quality

1. **Correctness** – Does it do what it should?
2. **Reliability** – Can it perform under expected conditions?
3. **Efficiency** – Optimal use of resources (CPU, memory).
4. **Integrity** – Security and protection from unauthorized access.
5. **Usability** – Easy to learn and use.
6. **Maintainability** – Easy to update, fix, or enhance.
7. **Portability** – Works in different environments.
8. **Testability** – Easy to test and validate.
9. **Flexibility** – Can adapt to changes.
10. **Reusability** – Components can be reused in other applications.

Role of Metrics and Measurement

Why Metrics?

- "You can't improve what you don't measure."
- Metrics provide **quantitative data** to assess software quality and process effectiveness.

Types of Metrics

1. Product Metrics

- Measure characteristics of the software product.
- Example: Size (LOC, Function Points), complexity (Cyclomatic Complexity), defect density.

2. Process Metrics

- Measure effectiveness of the software process.
- Example: Effort (person-hours), schedule variance, defect removal efficiency.

3. Project Metrics

- Focus on project management and progress.
- Example: Cost variance, productivity, team performance.

Examples of Important Metrics

- **LOC (Lines of Code):** Size measure.
- **Function Points:** Measures functionality delivered.
- **Cyclomatic Complexity (McCabe):** Measures control flow complexity.
- **Defect Density:** No. of defects per KLOC.
- **Mean Time to Failure (MTTF):** Reliability measure.

Benefits of Metrics

- Provides objective basis for decision-making.
- Helps track progress (time, cost, effort).
- Supports process improvement (CMMI, ISO 9001, Six Sigma).
- Improves quality by identifying weak areas.

Link Between CMM, Software Quality, and Metrics

- CMM Levels 4 & 5 rely heavily on quantitative measurement & metrics.
- **Software Quality** cannot be assured without **measurement** (you need metrics to evaluate correctness, reliability, etc.).
- **Metrics** → **Improve Quality** → **Achieve Higher Maturity in CMM.**

Software Quality:

Software Quality = *The degree to which a software product meets user requirements, expectations, and established standards.*

- It is not just about **working software**, but about **how well it works** under different conditions, and how easy it is to **maintain, use, and trust**.
- ISO 9126 / ISO 25010 defines software quality in terms of **quality attributes**.

Importance of Software Quality

- Reduces cost of rework and maintenance.
- Improves user satisfaction.
- Enhances reliability and trust.
- Helps meet industry standards (ISO 9001, CMMI, Six Sigma).
- Essential in critical domains (healthcare, aviation, defense, banking).

Dimensions / Factors of Software Quality

McCall's Quality Model (1977)

Defines three categories:

1. **Product Operation** → Correctness, Reliability, Efficiency, Integrity, Usability.
2. **Product Revision** → Maintainability, Flexibility, Testability.
3. **Product Transition** → Portability, Reusability, Interoperability.

ISO 25010 Software Quality Model

Defines **8 Quality Characteristics** (with sub-characteristics):

1. Functional Suitability

- Functional completeness
- Functional correctness
- Functional appropriateness

2. Performance Efficiency

- Time behavior (response time, throughput)
- Resource utilization
- Capacity

3. Compatibility

- Co-existence (works with other software on same environment)
- Interoperability (ability to exchange information with other systems)

4. Usability

- Learnability, Operability, Accessibility
- User interface aesthetics

5. Reliability

- Maturity, Availability, Fault tolerance, Recoverability

6. Security

- Confidentiality, Integrity, Non-repudiation, Authenticity

7. Maintainability

- Modularity, Reusability, Analyzability, Modifiability, Testability

8. Portability

- Adaptability, Installability, Replaceability

Software Quality Assurance (SQA)

SQA = A set of activities that ensures **processes and products conform to quality standards**.

Objectives of SQA

- Build confidence in software quality.
- Ensure compliance with standards.
- Prevent defects (not just detect).

SQA Activities

- Reviews (requirement review, code review).
- Audits (process audit, product audit).
- Testing (unit, integration, system, acceptance).
- Standards enforcement (ISO, IEEE, CMMI).

5. Software Quality Control (SQC)

- Subset of SQA.
- Involves **testing activities** to detect defects.
- Focused on product quality rather than process quality.

SQA = **Prevention** (process-oriented)

SQC = **Detection** (product-oriented)

6. Software Quality Metrics

Metrics help measure **quality quantitatively**.

Examples:

1. Reliability Metrics

- Mean Time to Failure (MTTF)
- Mean Time to Repair (MTTR)
- Availability = $MTTF / (MTTF + MTTR)$

2. Complexity Metrics

- Cyclomatic Complexity (McCabe's metric)
- Halstead's metrics

3. Size Metrics

- LOC (Lines of Code)
- Function Points

4. Defect Metrics

- Defect density = $\text{No. of defects} / \text{KLOC}$
- Defect removal efficiency (DRE)

Cost of Software Quality (CoSQ)

1. **Prevention Costs** → Cost of training, standards, reviews.

2. **Appraisal Costs** → Cost of inspection, testing.

3. **Failure Costs** →

- **Internal failure:** Defects found before release.
- **External failure:** Defects found after release (very costly).

Challenges in Ensuring Software Quality

- Changing customer requirements.
- Lack of communication between stakeholders.
- Trade-off between cost, time, and quality.
- Measuring non-functional attributes (usability, reliability).
- Distributed and agile development environments.